

中山大学集成电路学院 攻读硕士/博士学位论文 开题报告

基于并行计算平台的超大规模集成电路高效全
局布线算法研究

**Researches on Efficient Global Routing
Algorithms for Very Large Scale Integrated
Circuits Based on the Parallel Computing
Platform**

题 目

姓 名 冯俊熙

学 号 23215413

指 导 教 师 冯浪

专 业 集成电路工程

研 究 方 向 电子设计自动化（EDA）

答 辩 日 期

一、选题背景及意义

1.1 选题背景

随着半导体技术的不断进步，超大规模集成电路（Very Large Scale Integration Circuit, VLSI）的设计复杂度逐渐提高，工艺制程已经从过去的微米级别发展到今天的纳米级别。集成电路（Integration Circuit, IC）制造的规模和密度指数增长，导致 VLSI 设计复杂度指数提高，由此也催生了电子设计自动化（Electronic Design Automation, EDA）软件的出现。如今，电子设计自动化软件已经成为集成电路产业中不可或缺的一部分，在 VLSI 全产业链中发挥重要作用。

在电子设计自动化的软件的组成部分中，最核心的是应用于 VLSI 设计流程中的特定算法。布线（routing）是集成电路设计的关键步骤之一，其目的是在满足设计规则和布局规则的前提下，将所有逻辑单元通过互连线连接起来，布线质量的优劣直接影响到芯片的性能-功耗-面积（Performance-Power-Area, PPA）。布线问题是 NP 难问题，涉及的变量众多、约束复杂。在布线过程中，设计人员不仅需要满足所有的连接需求，还需要优化互连的电气特性，例如减少信号延迟、降低功耗、避免干扰等。当前主流的布线算法分为两个层次的布线：全局布线（Global Routing）和详细布线（Detailed Routing）。全局布线的任务是在有限的时间内，粗粒度地找到版图中所有线网（Net）的金属线连接方式及层数分配，为后续的细节布线（Detail Routing）流程提供关键引导，为每一条连接生成一个大致走线路径，确保所有的连接需求在宏观上可以满足。全局布线通常以减少线长（Wirelength）、通孔（Via）数、及拥塞（Congestion）量为优化目标。详细布线则负责在更精细的层次上处理实际的布线细节，以确保信号能够通过物理互连线正确传输。

随着现代 IC 设计规模的扩大，全局布线问题逐渐呈现出以下几个主要挑战：

- 1) 规模巨大：当前的先进芯片设计可能包含数十亿甚至超过百亿个晶体管，例如苹果公司最新推出的 M4 处理器就包含高达 280 亿个晶体管。传统的基于 CPU (Central Processing Unit, 中央处理单元) 的布线算法过去在处理小规模电路布线问题时表现良好，但受摩尔定律影响，集成电路中晶

晶体管规模随时间推进会指数上升。面对如此海量的晶体管数量，目前基于 CPU 的布线算法逐渐表现出无法兼顾设计速度与设计质量的瓶颈。

- 2) 结构复杂：伴随着晶体管规模的增加，晶体管与晶体管之间的拓扑连线的复杂度也大幅增加，布线问题的复杂度随晶体管规模增长呈现非线性增长。不仅布线路径会变得越来越复杂，而且处理布线场景中产生的拥塞也越来越困难。
- 3) 约束变多：由于集成电路的工艺制程已经下探到 3 纳米左右，集成电路制造端面临越来越多的物理层面的限制，反映到布线问题中就对应了繁多且复杂的数学约束。

以上列举的种种挑战，导致传统的全局布线算法无法满足电路规模日益增长的背景下对速度的需求，并且由于布线问题中不断增长的电路连接复杂度，非全局视角的传统布线算法可能无法获得全局最优解。因此，目前在电子设计自动化后端领域，开发高效的全局布线算法一直是最重要的核心问题之一。

1.2 选题意义

传统的全局布线算法，如全局布线算法中的 CUGR[1]、SPRoute[2]等算法均基于 CPU 平台运行。尽管在处理小规模电路时表现良好，但在大规模集成电路设计中，计算时间和内存消耗呈指数增长，导致随着现代集成电路规模不断扩大，自动化设计效率与可扩展性受限。同时，传统的全局布线算法从算法流程的视角来看，其布线的做法是将一个大的全局的布线问题分解成需要串行解决的众多局部的小问题，受其串行处理的算法逻辑限制，布线过程中只能针对局部的小问题求取最优解，难以在全局的视角下获得全局的最优解。传统串行布线算法尽管在处理布线资源充沛的小规模简单电路中表现良好，但在互连情况更加复杂、布线约束更多的大规模电路中，可能会出现更多的拥塞情况难以纾解的情况，无法获得令人满意的布线结果。近年来，为了缓解布线问题所面临的挑战，基于并行计算平台的并行全局布线算法逐渐受到关注。

并行计算平台如 GPU（Graphics Processing Unit，图形处理单元）相对于 CPU，由于并行计算单元众多的特性具有数据吞吐量大的优势，如果能设计出一款计算资源利用效率高的并且能同时在 CPU 平台和 GPU 平台跨平台运行的布

线程序，将可以显著加速超大规模集成电路全局布线中某些计算密集型任务，相比于仅在 CPU 上运行的算法，可以利用 GPU 强大计算资源的算法在大规模电路设计布线过程中在运行时间等方面能获得可观的速度优势。

并行布线算法相对于串行布线算法，由于能同时处理多个线网，不仅能综合衡量每个线网布线方案的优劣，获得更好的全局布线结果，而且可以契合 GPU 等并行计算平台的并行计算优势，在速度和效果上均取得相对于传统串行布线算法的优势。

在基于并行计算平台的并行全局布线算法中，并行计算的应用可以分为以下两个方面：

- 1) 数据级并行：全局布线算法涉及大量的数据计算以及处理等任务，例如全局布线资源的并行更新、多路径并行搜索等。这些任务具有很强的并行性，适合在 GPU 等大规模并行处理器上执行。GPU 的优势在于其能够并行处理大量小规模的计算任务，因此在处理诸如全局布线资源的并行更新、多路径并行搜索等操作时表现出色。
- 2) 线网级并行：通过建立统一的布线时所使用的数据结构，可以使全局布线中多个线网同时进行布线。比如可以通过采用多线程等手段同时处理多个无因果关系的布线任务，可以获得不影响全局最优的布线方案；再比如可以通过将完整布线问题进行数学建模然后结合深度学习工具采取连续化概率选择的布线方案，提高全局布线效率。此外，一些基于迭代优化，逐步逼近全局最优布线的算法也可以嵌入到整个并行全局布线流程中，进一步优化布线的效果。

表 1 并行层级及其主要作用

并行层级	主要作用
数据级并行	利用 GPU 平台的数据吞吐量大优势提升全局布线速度
线网级并行	获得全局更优的全局布线结果

本选题的研究具有重要的重要意义，数据级和线网级这两个层级的并行能够在布线速度和布线质量上均取得进步。基于并行计算平台的并行全局布线算法不仅有助于解决传统全局布线算法在处理大规模集成电路设计中的瓶颈问题，在芯片后端设计中缩短设计周期，满足业界对快速上市的要求；还能够为未来

的电子设计自动化工具提供新的全局布线算法设计思路，推动 AI4EDA（人工智能应用于电子设计自动化）相关的技术进步。

二、研究现状

近年来，国内外学者们针对全局布线算法以及并行计算做了大量研究。通常来说，全局布线可以分为生成树、模板布线、迷宫布线三个阶段，如图 1 所示：

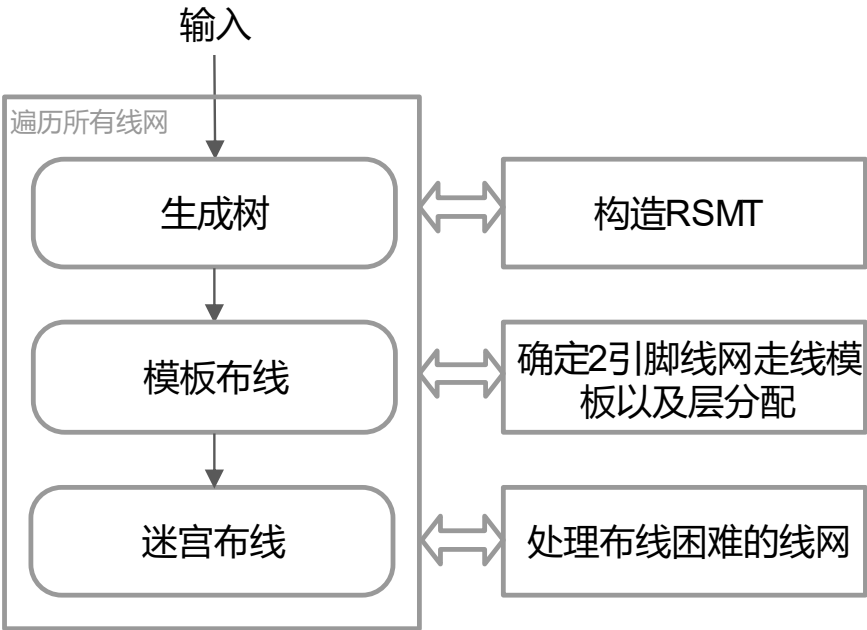


图 1 全局布线流程

1. 生成树（Tree Generation）：利用 FLUTE[3]等算法快速生成一个直角斯坦纳最小树（Rectilinear Steiner Minimal Tree, RSMT）。FLUTE 是一种采用查找表方法的直角斯坦纳最小树构造算法，该算法速度快且可以在低度（low-degree，即网络中引脚数目）线网上取得最优结果。直角斯坦纳最小树是指在二维平面上连接一组引脚的最短路径。通过直角斯坦纳最小树可以快速赋予线网初步的布线参考。

2.模板布线（Pattern Routing）：对于第一步所生成的直角斯坦纳最小树，利用深度优先搜索（Depth First Search, DFS）算法将直角斯坦纳最小树拆解成 2 引脚线网（two-pin net），2 引脚线网指树（Tree）中每两个共享一条边的两点所组成的线网。由于直角斯坦纳最小树仅仅只是二维的，需要利用动态规划（Dynamic Programming）等算法来决定 2 引脚线网的具体走向以及金属线的层

分配 (layer assignment)。即使用符合某些模板(如 L 型或 Z 型)的走线模板 (Pattern) 替换直角斯坦纳最小树的每个 2 引脚线网同时进行层分配从而快速的完成 3D 的走线。

3. 迷宫布线 (Maze Routing)：经过以上的算法流程，可能仍然存在难以布线的线网，采用迷宫布线算法可以较好的完成对这些线网的布线。

需要注意的是，在撤销重连阶段和迷宫布线阶段，这两个阶段仅针对前一个阶段后仍然拥塞的线网进行重新布线。并且，越前期的阶段对最终布线结果造成的影响越深远，因为从计算粒度粗细的角度来衡量整个全局布线算法流程，其计算粒度随布线流程机型变得越来越细，前期粗粒度的布线效果不好就会导致后期细粒度的布线任务负荷较大以及修正空间受限继而导致布线速度与布线效果的双双变差。例如模板布线阶段仅仅考虑有限数量的走线路径，所需的计算资源较少，而迷宫布线阶段则需要考虑版图上所有可走线网格所能构成的走线路径，所需的计算资源大大增加。如果模板布线阶段的效果不好，则会导致进入迷宫布线阶段的线网数量增多，一方面耗时会大大增加，另一方面由于布线后期版图上所剩布线资源不多导致迷宫布线能够进行的修正空间受限继而最终的布线结果也会变差。

在树生成阶段，FLUTE 算法是一个非常经典的直角斯坦纳最小树的生成算法，然而由于 FLUTE 算法进行树生成的时候仅考虑线长而不考虑后续布线阶段可能出现的拥塞现象，所以 FLUTE 算法树生成的结果会导致达成全局最优布线的可能性受限。基于前述原因，近年来出现了一系列关于树生成算法或是快速高精度拥塞预测算法的研究。FastRoute[4]中提出了一种可以感知拥塞和通孔的算法，可以依据布线区域的拥塞图映射出新的引脚位置并结合 FLUTE 算法生成直角斯坦纳最小树，在拥塞方向和拥塞区域使用布线需求较少的树拓扑结构。Guo[5]等学者提出了一种基于规则的高效构建避障直角斯坦纳最小树算法，通过引入特定情况的绕线规则，从而在增加很少时间开销的前提下有效的构建了能减少拥塞的直角斯坦纳最小树。在 GPU 加速生成树方面，Zizheng[6]等学者针对生成树阶段中耗时占比最多的少部分多引脚（指引脚数大于等于 10）线网，通过拆分线网为多个少引脚（指引脚数不大于 10）线网的方式获得可利用 GPU 并行加速的子线网数据结构从而使该算法获得 GPU 加速。此外还有 SALT[7]算

法、TreeNet[8]算法等算法针对有信号源的线网结构的生成树算法进行了优化，优化了走线延时。

在模板布线阶段，算法需要快速根据预先设定的走线模板快速决定最终的布线方案。其中有通过 2D 的方式来实现模板布线的，如 FastRoute 4.0[9]、NCTU-GR 2.0[10]、SPRoute 2.0[11]，这些方案的做法是通过将多层的版图在进行模板布线时压缩成单层的来简化问题，在 2D 的情形下得到布线解决方案以后，再通过层分配技术将 2D 的布线结果映射到 3D 的版图上去。然而，这种 2D 的布线算法由于缺失了层维度上的信息，映射到 3D 空间中时，可能会出现布线资源分散在不同层上从而无法避免 congestion 的情况。近年来越来越多的方案选择了通过 3D 的方式实现模板布线，例如 CUGR[12]、CUGR 2.0[13]、FastGR[14]、DGR[15]，它们会在 3D 情形下直接进行布线而不需经过 2D 和 3D 之间的映射转换过程。CUGR 利用动态规划算法同时处理 2 引脚走线布线模板选择问题和层分配问题，可以在局部视角下获得最优的 3D 模板布线结果。同时它还引入了多线程技术来实现并行加速，通过边界框 (BoundingBox) 是否重叠来判断线网与线网之间是否相关，若不相关便可利用多线程同时布线而不影响最终结果的质量。它的迭代版本 CUGR 2.0 针对模板布线中由于走线模板单调而出现的 congestion 采取了撤销重连的方法，通过边移动技术在发生 congestion 的线网上创造可以绕过 congestion 的有向无环图(Directed acyclic graph, DAG)在较少增加时间的前提下实现了模板布线效果的有效提升，同时由于它能在模板布线阶段就解决大部分的 congestion 而不至于将 congestion 问题延后到更慢的迷宫布线阶段，因此它仅需一个线程就在布线速度和质量两方面取得了很好的权衡。FastGR 重点针对 2 引脚线网进行研究，实现了并行的 L 形模板布线。DGR 则采取一种全新的模型，作者首先建模树森林，树森林对应两个层次的选择概率矩阵，第一个层次的选择概率矩阵表示每个线网中不同树拓扑的选择概率；第二个层次的选择概率矩阵表示所有树拓扑拆分出来的每个 2 引脚线网中不同布线模板的选择概率。然后作者建模线长、通孔、congestion 的权重。最后，并行累加每个线网中每种布线选择造成的开销乘以该特定布线选择对应的选择概率，从而获得一个目标函数表达式。完成以上建模工作以后，利用优化器去优化目标函数的最小值，形成更小目标函数值的权重参数对应了更优的全局布线选择。由于可以利用 GPU 的并行计算资源，从而

使这种算法实现了并行计算。这种利用概率来连续化优化目标表达函数的机制减少了布线时间并提升了布线质量。

在迷宫布线阶段，可以牺牲一定程度的线长与通孔指标以尽可能减少前面阶段中所出现的拥塞，是考虑粒度最细也是最后的布线阶段。NCTU-GR 2.0 在有界长度迷宫布线(bounded-length maze routing, BLMR)问题中提出了两种方法，第一种是最优方法，第二种是启发式演化法，尽管第二种方法不能保证最优解，但速度比第一种快得多。同时，NCTU-GR2.0 引入多线程机制加速迷宫布线过程。CUGR 采取两个不同考虑粒度的层次来实现速度和效果的均衡，第一层是粗粒度迷宫布线，第二层是细粒度迷宫布线，在粗粒度迷宫布线中，为了缩小搜索范围以简化问题，作者将版图划分为粗粒度的网格，在粗粒度网格上利用迷宫布线算法先得到一个宽大的路径，意味着最终走线一定在这个宽大的路径范围之内，然后将这个宽大路径结果交给细粒度迷宫布线寻找具体的布线路径，由于缩小了路径搜索范围，即使是细粒度的迷宫布线算法也可以较为快速的找到布线方案。CUGR2.0 则采取了一种稀疏网格的方式实现了路径搜索空间的减少，同时叠加候选路径偏移的方法使迷宫布线的走线方案在版图上分布更均匀从而减少拥塞情况发生。SPRoute在迷宫布线阶段中通过多线程实现了线网级的并行算法，实现了细粒度路径的并行搜索。Han 等人[16]提出了基于 GPU 的 A* 路径搜索算法，A*路径搜索算法是一种比迷宫布线中惯常使用的水波法更高效的算法，结合 GPU 的加速作用可以获取更快的迷宫布线速度。GAMER[17]提出了一种新的在规则网格图上通过并行扫描技术来实现并行加速的迷宫布线算法，由此有效加速了这一阶段的布线速度。

通过对以上研究的总结，可以发现当前该领域对于基于并行平台的并行全局布线算法的研究仍然不足，有的算法通过巧妙的方式实现了 GPU 加速，但无法实现多个线网的同步布线。有的算法可以在某一个布线阶段实现线网级的并行，但是仍然无法提前进行拥塞感知。因此想要实现一种快速的低拥塞发生率的全局布线算法，依然具有挑战性。

三、研究内容、实施方案

3.1 研究内容

在全局布线过程中，布线器需要在有限的时间内，粗粒度地找到版图中的所有线网的金属线连接方式及层数分配，为后续的细节布线流程提供关键引导。全局布线通常以减少线长、通孔数、及拥塞量为优化目标。随着芯片设计复杂度以及设计规模的增加，一方面，传统的算法不能获得令人满意的布线速度，难以利用并行计算平台对布线算法进行加速，另一方面，传统的全局布线算法因为对版图整体布线资源及线网分布的全局考虑仍然不充分，难以在布线前期非常有效地应对设计中出现的拥塞问题，极大影响了后续 EDA 算法的运行速度与最终版图的质量。

针对以上提出的问题，本文意图针对布线流程中的多个布线阶段都尽可能实现利用并行计算平台加速的并行布线算法。本文希望结合其他已有算法的优点的前提下在以下几个方面对全局布线算法展开研究：

- 1) 实现拥塞感知的并行生成树算法。由于前期树生成质量对后续布线影响深远，所以在越前期的阶段就需要尽可能生成可以有效避免拥塞的树拓扑同时尽可能少的牺牲线长等其他指标。
- 2) 实现细粒度的模板布线并行算法。利用 DGR 的方法结合上一阶段产生的树拓扑实现更优的并行全局布线结果。同时可以引入 Z 形、U 形等考虑粒度更细的走线模板实现更少的拥塞发生。
- 3) 利用反馈迭代的方式实现更精准高效的并行撤销重连策略。针对难以避免拥塞的线网，通过逐次的撤销重连策略逐步更改树拓扑，从而逼近全局最优。
- 4) 针对迷宫布线采用并行扫描/多线程等方式实现并行加速。结合 GAMER 以及 SPRoute 的设计更加高效的迷宫布线算法。

3.2 实施方案

(1) 研究方法

本研究所需要的研究方法主要包括算法设计、代码编写、实验测试等三个步骤。

算法设计是整个研究过程中的核心环节，它直接决定了最终算法的效果如何。在这一部分，研究将围绕布线问题的特性，结合同行计算平台的优势，设计一个能够处理超大规模集成电路复杂布线问题的高效并行算法。

代码编写部分需要按照设计好的算法进行实际的实现。此阶段的核心任务是按照并行算法的设计要求去构建相关的数据结构，利用一些已有的包工具实现对 GPU 算力资源的高效利用。

实验测试是验证基于并行平台的并行全局布线算法有效性的重要环节。在这一部分，需要选择合适的测试基准，通过大量的实验数据分析，评估并行布线算法的性能，并根据测试结果对算法或者代码进行反馈修改。

(2) 技术路线

本研究在实现算法的时候，针对并行算法的技术路线，可以考虑通过 PyTorch、TensorFlow 等现成的机器学习套件来实现并行，也可以考虑针对布线算法中特定的流程进行 CUDA 编程以求充分利用 GPU 的并行计算能力。

四、研究条件及进展

4.1 研究条件

由于针对的是大规模集成电路布线问题，因此本课题的研究需要一定的硬件资源：包括高性能的 CPU 计算平台以及 GPU 计算平台。其中会利用多线程、机器学习等方式去提升并行全局布线算法的表现。

此外，还需要相应的软件资源。需要使用编程语言如 C++、Python、CUDA 等来实现对硬件资源的效率利用。在测试过程中还需要权威的测试基准来实现对并行布线算法的全面测试。为了建立统一的衡量对比标准，可能还需要 EDA 软件例如 Cadence 厂的 Innovus[18]，针对最终布线方案利用测试基准所提供的衡量标准来判定布线算法的优劣。

4.2 目前进展

目前已经针对“反馈迭代实现更高效的撤销重连”进行了研究，相关专利申请书中。

五、论文目标及计划

5.1 学位论文预期创新点

(1)提出拥塞感知的并行生成树算法：在全局布线前期获取能有效避免拥塞的树拓扑

(2)实现细粒度的模板布线并行算法：针对大规模集成电路的布线问题，通过并行布线能力快速完成模板布线。

(3)利用反馈迭代的方式实现更精准高效的并行撤销重连策略：通过逐步迭代使布线结果尽可能逼近全局最优。

(4)利用 GPU 对迷宫布线实现并行加速：加快迷宫布线的速度。

5.2 学位论文预期目标

论文与专利：本课题拟整理课题研究，并对研究内容做出总结并至少发表论文 1 篇，专利 1 篇。

5.3 学位论文进度安排

(1) 时间：2023 年 09 月至 2024 年 07 月

完成相关课程的学习以及论文的调研；

(2) 时间：2024 年 07 月至 2025 年 07 月

完成相关课题研究以及工程实施

(3) 时间：2025 年 07 月至 2026 年 01 月

完成进行中期答辩，论文撰写等工作

(4) 时间：2026 年 01 月至 2026 年 06 月

完成毕设盲审，论文答辩等工作

参考文献:

- [1] J. Liu, C.-W. Pui, F. Wang, and E. F. Young, “Cugr: Detailed-routabilitydriven 3d global routing with probabilistic resource model,” in 2020 57th ACM/IEEE Design Automation Conference (DAC). IEEE, 2020, pp. 1–6.
- [2] J. He, M. Burtcher, R. Manohar, and K. Pingali, “Sproute: A scalable parallel negotiation-based global router,” in 2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD). IEEE, 2019, pp. 1–8.
- [3] C. Chu and Y. -C. Wong, "FLUTE: Fast Lookup Table Based Rectilinear Steiner Minimal Tree Algorithm for VLSI Design," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 27, no. 1, pp. 70-83, Jan. 2008.
- [4] M. Pan and C. Chu, “Fastroute: A step to integrate global routing into placement,” in Proceedings of the 2006 IEEE/ACM international conference on Computer-aided design, pp. 464–471, ACM, 2006.
- [5] J. Guo, H. Kong and L. Feng, "A Rule-Based High Efficient Obstacle-Avoiding RSMT Algorithm for VLSI Routing," 2024 IEEE International Symposium on Circuits and Systems (ISCAS), Singapore, Singapore, 2024, pp. 1-5.
- [6] Z. Guo, F. Gu and Y. Lin, "GPU-Accelerated Rectilinear Steiner Tree Generation," 2022 IEEE/ACM International Conference On Computer Aided Design (ICCAD), San Diego, CA, USA, 2022, pp. 1-9.
- [7] G. Chen and E. F. Young, “Salt: provably good routing topology by a novel steiner shallow-light tree algorithm,” IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, no. 6, pp. 1217–1230, 2019.
- [8] W. Li, Y. Qu, G. Chen, Y. Ma, and B. Yu, “TreeNet: Deep point cloud embedding for routing tree construction,” in Proceedings of the 26th Asia and South Pacific Design Automation Conference, 2021, pp. 164–169.
- [9] Y. Xu, Y. Zhang, and C. Chu, “Fastroute 4.0: Global router with efficient via minimization,” in 2009 Asia and South Pacific Design Automation Conference. IEEE, 2009, pp. 576–581.
- [10] Wen-Hao Liu, Wei-Chun Kao, Yih-Lang Li, and Kai-Yuan Chao. 2013. NCTU-GR

2.0: Multithreaded collision-aware global routing with bounded-length maze routing. *IEEE Transactions on computer-aided design of integrated circuits and systems* 32, 5 (2013), 709–722.

- [11] Jiayuan He, Udit Agarwal, Yihang Yang, Rajit Manohar, and Keshav Pingali. 2022. SPRoute 2.0: A detailed-routability-driven deterministic parallel global router with soft capacity. In 2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC). 586–591.
- [12] Young E . CUGR: detailed-routability-driven 3D global routing with probabilistic resource model[C]// 2020 57th ACM/IEEE Design Automation Conference (DAC). ACM, 2020.
- [13] J. Liu and E. F. Young, “EDGE: Efficient DAG-based Global Routing Engine,” in 2023 60th ACM/IEEE Design Automation Conference (DAC). IEEE, 2023.
- [14] Siting Liu, Peiyu Liao, Rui Zhang, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. FastGR: Global Routing on CPU-GPU with Heterogeneous Task Graph Scheduler. In 2022 Design, Automation Test in Europe Conference Exhibition (DATE).
- [15] Wei Li, Rongjian Liang, Anthony Agnesina, Haoyu Yang, Chia-Tung Ho, Anand Rajaram, Haoxing Ren, “DGR: Differentiable Global Router”, ACM/IEEE Design Automation Conference (DAC), San Francisco, June 23-27, 2024.
- [16] Yiding Han, Koushik Chakraborty, and Sanghamitra Roy. 2013. A global router on GPU architecture. In 2013 IEEE 31st International Conference on Computer Design (ICCD). 78–84.
- [17] Shiju Lin, Jinwei Liu, and Martin D.F. Wong. 2021. GAMER: GPU Accelerated Maze Routing. In 2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD). 1–8. <https://doi.org/10.1109/ICCAD51958.2021.9643563>
- [18] “Cadence innovus implementation system.” <https://www.cadence.com>.